

Real-Time Event Ingestion Pipeline Architecture

SYSTEM ARCHITECTURE SPECIFICATION

Component: Real-Time Event Ingestion Pipeline Author: Julian Sterling, Lead Architect Date: August 17, 2026

1. Introduction This specification outlines the proposed architecture for the new real-time event ingestion pipeline for the Starlight Networks telemetry platform. The goal is to replace our monolithic REST-based ingestion service with a highly scalable, decoupled, event-driven architecture capable of processing 50,000 events per second with sub-second latency.

2. Core Components The new pipeline will leverage the following technology stack:

- Ingestion Gateway: A fleet of stateless Go-based microservices running in a Kubernetes cluster. These nodes will receive incoming JSON payloads from IoT edge devices, perform basic schema validation, and authenticate the payload using JWT tokens.
- Message Broker: We will utilize an Apache Kafka cluster as the central nervous system. The Gateway services will act as producers, publishing validated events to partitioned Kafka topics based on the event type (e.g., 'telemetry.temperature', 'telemetry.voltage').
- Stream Processing: Apache Flink will be deployed to consume the raw event streams from Kafka. Flink jobs will handle stateful aggregations, anomaly detection (e.g., identifying temperature spikes over a rolling 5-minute window), and data enrichment.
- Persistent Storage: Processed and enriched data will be sunk into a Cassandra database for low-latency operational querying, while raw events will be archived in AWS S3 for long-term compliance storage and batch machine learning training.

3. Scalability and Resilience The Ingestion Gateway will be deployed behind a Layer 7 load balancer equipped with auto-scaling triggers based on CPU utilization. The Kafka cluster will be provisioned across three availability zones to ensure fault tolerance. In the event of a downstream storage failure, Flink will rely on its internal checkpointing mechanism to guarantee exactly-once processing semantics without data loss.

4. Next Steps The engineering team will begin provisioning the staging environment infrastructure next sprint. A proof-of-concept load test targeting 10,000 EPS is scheduled for the end of the month.